C++ Coroutine TS Issues

Doc. no.	P0664R7
Revises	P0664R6
Date:	Revised 2019-01-16
Project:	Programming Language C++
Reference:	ISO/IEC TS 22277, C++ Extensions for Coroutines
Audience:	CWG, EWG, LWG
Reply to:	Gor Nishanov <gorn@microsoft.com>

Introduction

All proposed resolutions wording is relative to N4775.

Previous issue list is P0664R6.

Table of content

LWG issues

31. Add a note warning about thread switching near await and/or coroutine_handle wording.

Status: Has wording **Submitter:** SG1 **Opened:** 2018-06-08

Issue:

Add a note warning about thread switching near await and/or coroutine_handle wording

Proposed wording

[This wording per SG1 also addresses issue 32]

Modify [coroutine.handle.resumption] as follows:

> **xx.xx.xx coroutine_handle resumption [coroutine.handle.resumption]**
>
> 1. Resuming a coroutine via `resume`, `operator()`, or `destroy` on an execution agent other than the one it was suspended on has implementation-defined behavior unless both are instances of `std::thread` or the thread that executes main.

[Note: a coroutine that is resumed on a different execution agent should also avoid relying on consistent thread identity throughout, such as holding a mutex object across a suspend point. *— End note.]*

[Note: A concurrent resumption of the coroutine via resume, operator(), or destroy may result in a data race. *—end note]*

```
void operator()() const;
void resume() const;
```

...
~~*[Note:* A concurrent resumption of the coroutine via resume, operator(), or destroy may result in a data race. *—end note]*~~

```
void destroy() const;
```

...
~~*[Note:* A concurrent resumption of the coroutine via resume, operator(), or destroy may result in a data race. *—end note]*~~

EWG issues

---

33. Parameter copy wording does not capture the intent.

Section: 11.4.4/11 [dcl.fct.def.coroutine] **Status:** Clarification Requested **Submitter:** Mathias Stearn **Opened:** 2018-06-09

[Clarification Requested by CWG - San Diego - 2018/11/07]

Issue:

The intent is that copies/moves of parameters (if required) are created preserving the exact type (including references, r-references, etc). The wording in 11.4.4[dcl.fct.def.coroutine]/11 does not seem to express that clearly.

Proposed wording:

Modify paragraph 11 of 11.4.4/[dcl.fct.def.coroutine] as follows:

> When a coroutine is invoked, a copy is created for each coroutine parameter. Each such copy is an object or reference with automatic storage duration and has the same type as the corresponding parameter. ~~that~~Each copy is direct-initialized ([dcl.init]) from ~~an lvalue referring to~~ the corresponding parameter ~~if the parameter is an lvalue reference, and from an xvalue referring to it otherwise~~. If the type of the copy is an rvalue reference type, then, for the purpose of this initialization the value category of the corresponding parameter is an rvalue. A ~~reference to~~use of a parameter in the function-body of the coroutine and in the call to the coroutine promise constructor is replaced by ~~a reference to~~ its copy. The initialization and destruction of each parameter copy occurs in the context of the called coroutine. Initializations of parameter copies are sequenced before the call to the coroutine promise constructor and indeterminately sequenced with respect to each other. The lifetime of parameter copies ends immediately after the lifetime of the coroutine promise object ends. [*Note:* If a coroutine has a parameter passed by reference, resuming the coroutine after

the lifetime of the entity referred to by that parameter has ended is likely to result in undefined behavior. *—end note*]

---

35. Remove `for co_await` statement.

Status: Active **Submitter:** Gor Nishanov **Opened:** 2018-01-16

Issue:

The wording `for co_await` statement makes assumptions of what future asynchronous generator interface will be. Remove it for now as not to constraint the design space for asynchronous generators.

Proposed wording change:

Strike sections [stmt.iter] and [stmt.ranged] from Coroutines TS working document.

CWG issues

---

34. Mandate the return type for return_void and return_value to be `void`.

Status: Active **Submitter:** Gor Nishanov **Opened:** 2018-01-16

Issue:

Mandate that the return type of `return_value` and `return_void` customization points to be of type `void` to leave room for possible future evolution.

Proposed wording attempt 1:

Modify the paragraph 2 of [stmt.return.coroutine] as follows:

> (2.1) -- *S* is *p*`.return_value(`*expr-or-braced-init-list*`)`, if the operand is a *braced-init-list* or an expression of non-`void` type;
>
> (2.1) -- *S* is `{` $expression_{opt}$ `; p.return_void(); }`, otherwise;
>
> *S* shall be a prvalue of type void. *p*`.return_void()` shall be a prvalue of type `void`.

Proposed wording attempt 2:

Add new paragraph after paragraph 3 of [stmt.return.coroutine]:

> 3. If *p*`.return_void()` is a valid expression, flowing off the end of a coroutine is equivalent to a *co_return* with no operand; otherwise flowing off the end of a coroutine results in undefined behavior.
>
> 4. The return type of `return_void` and `return_value` shall be `void`.

Editorial issues

28. Simplify stable name for Coroutines to be [def.coroutine]

Section: 11.4.4 [dcl.fct.def.coroutine] **Status:** Ready **Submitter:** Gor Nishanov **Opened:** 2018-03-10 **Last modified:** 2018-03-10

Proposed wording:

[~~dcl.fct.~~def.coroutine]

Not ready issues

---

26. Relax requirements on a coroutine_handle passed to `await_suspend`

Section: 8.3.8 [expr.await] **Status:** Waiting for more information **Submitter:** Gor Nishanov **Opened:** 2018-03-10 **Last modified:** 2018-11-10

[EWG Approved Rapperswil-2018/06/09]

[San Diego-2018/11/10: Alternative resolution for this issue is proposed. Waiting for details.]

Issue:

One of the implementation strategies for coroutines is to chop original function into as many functions (parts) as there are suspend points. In that case, it is possible for a compiler create a unique per suspend per function `coroutine_handle` which `resume` and `destroy` members can be direct calls to corresponding parts of the function.

Though no compiler is doing it now, we can allow implementors to experiment with this approach by relaxing the requirement on the `coroutine_handle` passed to `await_suspend`.

Proposed wording:

Add underlined text to 8.3.8/3.5:

> (3.5) — h is an object of type convertible to std::experimental::coroutine_handle<P> referring to the enclosing coroutine.

---

24. The specification of initial suspend point does not correctly captures the intent

Section: 11.4.4 [dcl.fct.def.coroutine] **Status:** Waiting for new wording **Submitter:** Gor Nishanov **Opened:** 2018-03-08 **Last modified:** 2018-05-05

[EWG Approved Rapperswil-2018/06/09]

[San Diego-2018/11/10: Alternative resolution for this issue is proposed. Waiting for details]

Issue:

Common use of the `initial_suspend` point in asynchronous coroutines is to suspend the coroutine during the initial invocation and post a request to resume the coroutine by a different execution agent. If an execution agent

at a later point cannot resume the coroutine, for example, because it is being shutdown, an error will be reported to a coroutine by resuming the coroutine and subsequently throwing an exception from `await_resume`.

Currently, the invocation of `initial_suspend` is specified in [dcl.fct.def.coroutine] as:

```
{
  P p;
  co_await p.initial_suspend(); // initial suspend point
  try { F } catch(...) { p.unhandled_exception(); }
final_suspend:
  co_await p.final_suspend(); // final suspend point
}
```

As specified, an exception from `await_resume` of `initial_suspend` will be thrown outside of the `try-catch` and will not be captured by `p.unhandled_exception()` and whoever waits for eventual completion of a coroutine will never learn about its completion.

This is a specification defect. The intent is to capture an exception thrown by `await_resume` of an awaitable returned by `initial_suspend` within the `try-catch` enclosing of the user authored body `F`.

The correct behavior has been implemented in MSVC staring with version 2015 and in clang trunk.

Proposed resolution:

Add underlying text to paragraph 11.4.4/3 as follows:

```
{
  P p;
  co_await p.initial_suspend(); // initial suspend point
  try {
    F
  } catch(...) { p.unhandled_exception(); }
final_suspend:
  co_await p.final_suspend(); // final suspend point
}
```

> except that any exception thrown after the *initial suspend point* and before the flow of execution reaches *F* also results in entering the *handler* of the *try-block* and, where an object denoted as *p* is the *promise object* of the coroutine and its type *P* is the *promise type* of the coroutine, ...

---

30. Re-enable coroutine return type deduction when coroutine `task` and `generator` types are available

Section: 10.1.7.4 [coroutine.handle] **Status:** Not ready **Submitter:** Gor Nishanov **Opened:** 2018-05-05 **Last modified:** 2018-05-05

Issue:

We stripped out automatic return type deduction for coroutines from N4499 in 2015. Put back the wording to do it once the appropriate types are available in the standard library.

Resolved issues

---

32. Add a normative text making it UB to migrate coroutines between certain kind of execution agents.

Status: Resolved **Submitter:** SG1 **Opened:** 2018-06-08 **Resolved:** 2019-01-16

Issue:

Add a normative text making it UB to migrate coroutines between certain kind of execution agents. Clarify that migrating between std::threads is OK. But migrating between CPU and GPU is UB.

[Resolved by the wording offered in issue 31 - 2019/01/16]